

Open Source & Cloud Native Ecosystem

10.1 Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa mampu:

1. Membedakan konsep *open source*, *cloud computing*, dan *cloud native*.
2. Menganalisis peran strategis *open source* sebagai fondasi ekosistem *cloud modern*.
3. Mengevaluasi model penerapan *open source* dalam arsitektur *hybrid-multi-cloud*.
4. Mengimplementasikan layanan *cloud* berbasis *open source* dengan pendekatan *self-hosted* dan *managed services*.
5. Membandingkan kelebihan, tantangan, dan tren *open source* di lingkungan *cloud*.

10.2 Open Source sebagai Fondasi Cloud Computing

Open source bukan sekadar alternatif, melainkan inti dari evolusi *cloud computing*. Mayoritas infrastruktur *cloud* publik (AWS, GCP, Azure) dibangun dan dioperasikan dengan perangkat lunak *open source*. Hubungan simbiosis ini terwujud dalam beberapa pola:

- *Infrastructure-as-Code (IaC) & Orchestration*:
Kubernetes (*container orchestration*), Terraform (*provisioning*), dan Ansible (konfigurasi) menjadi standar *de facto* untuk mengelola sumber daya *cloud*.
- *Cloud Native Foundation (CNCF) Landscape*:
Lebih dari 95% proyek dalam *cloud native stack* bersifat *open source* (contoh: Prometheus, Grafana, Envoy, Istio).
- *Model Bisnis Open Core*:
Perusahaan menawarkan versi dasar yang *open source*, dengan fitur premium berbayar (contoh: GitLab, Elasticsearch).

10.3 Konsep Self-Hosted vs. Managed Services di Cloud Open Source

A. Self-Hosted (DIY/On-Premise)

Organisasi mengelola penuh *stack* perangkat lunak *open source* pada infrastruktur milik sendiri atau *cloud* yang disewa.

- Contoh Implementasi:
 - Nextcloud untuk *file sharing & collaboration*.
 - OpenStack untuk *private cloud* IaaS.
 - Kubernetes on *bare-metal*/VMs untuk *container orchestration*.
- Keunggulan:
 - Modifikasi kode dan arsitektur sesuai kebutuhan.
 - Data tetap dalam yurisdiksi hukum organisasi.
 - Tidak ada *vendor lock-in* atau kenaikan biaya lisensi tak terduga.

- Dapat terhubung langsung dengan sistem *legacy*.
- Tantangan:
 - Membutuhkan tim DevOps/SRE yang kompeten.
 - *Patch management*, *hardening*, dan *compliance* sepenuhnya menjadi tanggung jawab internal.
 - Membutuhkan perencanaan kapasitas dan arsitektur yang matang.
 - Lebih lambat dibandingkan menggunakan layanan terkelola.

B. Managed Services (Cloud Provider)

Penyedia *cloud* menawarkan produk *open source* sebagai layanan terkelola (*as-a-Service*). Pengguna mengonsumsi layanan tanpa mengelola infrastruktur dasarnya.

- Contoh Layanan:
 - Amazon RDS for PostgreSQL/MySQL (*Database*)
 - Google Kubernetes Engine (GKE) (*Container Orchestration*)
 - Azure Managed Grafana (*Observability*)
- Keunggulan:
 - Penyedia mengelola pemeliharaan, *backup*, *patching*, dan *scaling*.
 - Keamanan infrastruktur dasar dijamin penyedia (*shared responsibility model*).
 - Dapat di-deploy di berbagai region dengan klik.
 - Terintegrasi langsung dengan ekosistem layanan *cloud provider* lainnya.
- Tantangan:
 - Meski menggunakan *open source*, migrasi antar penyedia bisa kompleks.
 - Tidak bisa memodifikasi *backend* atau versi perangkat lunak secara mendalam.
 - Biaya operasional berbasis konsumsi yang bisa sulit diprediksi.

10.4 Tujuan & Strategis Penerapan Open Source di Cloud

Penerapan *open source* di *cloud* didorong oleh tujuan strategis berikut:

1. Komunitas *open source* bergerak lebih cepat daripada pengembangan *proprietary*, memungkinkan adopsi teknologi terbaru (e.g., *service mesh*, *serverless framework*).
2. Arsitektur berbasis *open source* (e.g., aplikasi di Kubernetes) dapat dijalankan di *multi-cloud* atau *hybrid cloud*, meningkatkan *leverage* negosiasi.
3. *Security through obscurity* digantikan dengan *security through transparency*. Kode yang dapat diaudit publik meningkatkan kepercayaan dan memungkinkan *community-powered vulnerability detection*.
4. Menggeser biaya dari lisensi (*CapEx*) ke operasional dan inovasi (*OpEx*), dengan fleksibilitas untuk memilih model *self-hosted* atau *managed*.
5. Memastikan interoperabilitas dan masa depan (*future-proofing*) investasi teknologi.

10.5 Tren Open Source dalam Cloud Native & AI

- **MLOps & AI/ML Stack:**
Projek seperti Kubeflow, MLflow, dan PyTorch menyediakan platform *open source* untuk mengembangkan dan men-deploy model AI di *cloud*, sering dijalankan di atas Kubernetes.
- **GitOps & Platform Engineering:**
ArgoCD dan Flux memungkinkan *continuous delivery* untuk infrastruktur dan aplikasi di *cloud* dengan prinsip Git sebagai *single source of truth*.
- **FinOps & Observability:**
OpenCost (integrasi dengan Kubernetes) dan OpenTelemetry (standar *observability*) menjadi kunci untuk mengelola biaya dan performa di lingkungan *cloud* yang dinamis.

10.6 Keuntungan *Open Source* di *Cloud*

1. Prototipe dan *deploy* aplikasi lebih cepat dengan *pre-built container images* dan *helm charts* dari komunitas.
2. Lebih mudah menemukan *talent* dengan keahlian di teknologi *open source* populer (Kubernetes, Docker, Terraform) dibandingkan solusi *proprietary* tertentu.
3. Akses ke dokumentasi, tutorial, dan *stack overflow* yang luas dari komunitas global.
4. Investasi dalam skill dan arsitektur *open source* tetap relevan meskipun berganti penyedia *cloud*.

10.7 Tantangan & Pertimbangan Kritis

1. Biaya tersembunyi bisa muncul dari pelatihan, integrasi, dan operasional yang kompleks.
2. Banyaknya pilihan distribusi (e.g., berbagai distro Kubernetes) dapat menyebabkan kebingungan dan *compatibility issues*.
3. Pergeseran lisensi (contoh: Elasticsearch dari Apache 2.0 ke SSPL) dapat memengaruhi strategi *cloud*. Perlu pemahaman mendalam tentang lisensi (Apache 2.0, GPL, MIT, SSPL, BSL).
4. Organisasi harus aktif memantau *Common Vulnerabilities and Exposures (CVE)* dan melakukan patching, terlepas dari apakah *self-hosted* atau menggunakan *managed service*.

10.8 Landscape Layanan & Proyek *Open Source* untuk *Cloud*

Kategori	Contoh Proyek	Deskripsi Singkat & Use Case
Private Cloud IaaS	OpenStack, Apache CloudStack	Membangun <i>cloud</i> pribadi mirip AWS EC2/S3 di <i>on-premise</i> data center.
Container & Orchestration	Kubernetes, Docker, Podman	Dasar dari aplikasi <i>cloud-native</i> dan <i>microservices</i> .
Service Mesh	Istio, Linkerd, Consul	Mengelola komunikasi, keamanan, dan observabilitas antar <i>microservice</i> .
Observability	Prometheus, Grafana, Loki, Jaeger	Memantau metrik, log, dan <i>trace</i> aplikasi di <i>cloud</i> .
CI/CD & DevOps	GitLab CI, Jenkins, ArgoCD, Tekton	Otomatisasi pipeline pengembangan dan <i>deployment</i> .

Kategori	Contoh Proyek	Deskripsi Singkat & Use Case
Storage & Database	Ceph, MinIO, PostgreSQL, Redis	Penyimpanan objek, blok, dan <i>database</i> yang <i>scalable</i> .
AI/ML & Data	Kubeflow, Apache Spark, Airflow	Platform <i>data engineering</i> dan <i>machine learning</i> di <i>cloud</i> .
Collaboration & SaaS	Nextcloud, Mattermost, Discourse	Alternatif <i>open source</i> untuk Google Workspace/Slack.

LANGKAH PRAKTIKUM

Persiapan

Memperbarui Sistem

Perbarui sistem dengan menjalankan perintah berikut:

```
sudo apt update && sudo apt upgrade -y
```

Instalasi LAMP

Instal dan konfigurasi LAMP (Linux, Apache, MySQL, PHP), karena Nextcloud memerlukan server web dan basis data. Jalankan perintah berikut:

```
sudo apt install -y apache2 mariadb-server php libapache2-mod-php php-cgi  
php-mysql php-pear php-phpseclib php-mysql php-mbstring php-zip php-gd  
php-curl php-common php-imagick php-gmp php-intl php-apcu
```

Unduh Nextcloud menggunakan **wget**:

```
wget https://download.nextcloud.com/server/releases/latest.zip
```

Ekstrak file yang telah diunduh dan pindahkan ke direktori **/var/www/html**:

```
sudo unzip latest.zip -d /var/www/html
```

Akses direktori Nextcloud:

```
cd /var/www/html
```

Ubah kepemilikan direktori Nextcloud:

```
sudo chown -R www-data:www-data nextcloud
```

Konfigurasi MySQL

Atur kata sandi root untuk basis data dengan menjalankan:

```
sudo mysql_secure_installation
```

Login ke MySQL menggunakan perintah berikut:

```
sudo mysql -u root -p
```

Membuat Basis Data dan Pengguna

Gantilah `username` dan `password` sesuai kebutuhan:

```
CREATE DATABASE home_server;
```

```
CREATE USER 'username'@'localhost' IDENTIFIED BY 'password';
```

Izinkan pengguna untuk mengakses basis data `home_server`:

```
GRANT ALL PRIVILEGES ON home_server.* TO 'username'@'localhost';
```

```
FLUSH PRIVILEGES;
```

```
exit
```

Konfigurasi Virtual Host

Buat konfigurasi virtual host dengan perintah berikut:

```
sudo nano /etc/apache2/sites-available/nextcloud.conf
```

Isi berkas dengan konfigurasi berikut:

```
<VirtualHost *:80>
    ServerName localhost

    ServerAdmin admin@example.com
    DocumentRoot /var/www/html/nextcloud

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

    <Directory /var/www/html/nextcloud/>
        Options +FollowSymlinks
        AllowOverride All

        <IfModule mod_dav.c>
            Dav off
        </IfModule>

        SetEnv HOME /var/www/html/nextcloud
        SetEnv HTTP_HOME /var/www/html/nextcloud
    </Directory>
</VirtualHost>
```

Simpan berkas dan aktifkan konfigurasi:

```
sudo a2ensite nextcloud.conf && sudo systemctl restart apache2
```

Konfigurasi Nextcloud

Buka browser dan akses <http://<YOUR-IP-PUBLIC>/nextcloud>

Petunjuk instalasi Nextcloud akan muncul. Selama proses instalasi, atur basis data MariaDB, akun admin, dan direktori penyimpanan data.

Selanjutnya, jalankan perintah berikut untuk membuat crontab baru yang akan digunakan untuk menjalankan skrip crontab Nextcloud:

```
sudo crontab -u www-data -e
```

Parameter `-u www-data` digunakan karena server web Apache2 berjalan di atas pengguna tersebut.

Tambahkan konfigurasi berikut ke file crontab:

```
*/5 * * * * php -f /var/www/html/nextcloud/cron.php
```

Simpan dan keluar dari file setelah selesai.